GraphMana: graph-native data management for population genomics projects

Jianfeng Mao[1*]

[1*][Department to be completed], [Institution to be completed], [City], [Country].

Corresponding author(s). E-mail(s): jianfeng.mao@gmail.com;

Abstract

Population genomics projects rely on fragmented file-based workflows that lose provenance and require full reprocessing when samples are added. GraphMana is a data management platform built on graph database technology that stores variant data as packed genotype arrays with pre-computed population statistics, enabling incremental sample addition, provenance tracking, cohort management, and export to 17 formats from a single persistent source. Applied to the 1000 Genomes Project (3,202 samples, 70.7 million variants), GraphMana completes a multi-phase project lifecycle—import, incremental addition, annotation, cohort extraction, and multi-format export—otherwise requiring ad hoc scripting across multiple tools.

Keywords: population genomics, data management, graph database, variant call format, genotype storage, reproducibility

Population genomics projects at the scale of hundreds to tens of thousands of samples face a growing data management challenge that existing tools do not address. As new sequencing batches arrive, file-based workflows require full regeneration of every downstream result—every VCF [9], every PLINK binary [5], every TreeMix [12] input, every site frequency spectrum for dadi [10]—because flat-file formats encode the complete sample set and cannot be extended in place. Provenance is tracked manually if at all; sharing data subsets across formats demands bespoke scripting; and annotations embedded in file headers require rewriting the entire file even when only metadata has changed. This coordination burden—not computation—has become the primary time sink for the thousands of research groups operating at this scale,

falling in a gap between single-investigator projects (where manual tracking suffices) and biobank-scale programs (where platforms such as Hail [4] provide managed infrastructure).

The scale and diversity of affected projects continues to grow. In human genetics, the UK Biobank has released whole-genome sequences for 490,640 participants, the NIH All of Us Research Program has surpassed 414,000 genomes, and the TOPMed consortium has sequenced over 180,000 individuals. In crop science, the 3,000 Rice Genomes Project resequenced 3,010 accessions across 89 countries, while similar efforts are underway in wheat, maize, soybean, and other staple crops. In livestock, the 1000 Bull Genomes Project has sequenced 2,703 cattle, with companion initiatives extending to buffalo, sheep, and aquaculture species. In ecology, the 1001 Genomes Project catalogued variation across *Arabidopsis thaliana*, and the Earth BioGenome Project has sequenced 3,000 eukaryotic species toward a goal of 1.67 million. These projects share a common trajectory: sample counts grow, new batches arrive continuously, and the analytical scope expands from variant calling to population structure, selection scans, functional annotation, and multi-format data sharing.

Consider the daily reality. On Monday, a sequencing core delivers 200 new samples; integrating them means regenerating every downstream file. On Tuesday, a collaborator requests an EIGENSTRAT [13] file filtered to European samples with MAF above 5%; the request requires a bespoke script whose parameters will not be recorded alongside its output. On Wednesday, a new ClinVar release arrives; updating annotations means rewriting the entire VCF, even though genotype data has not changed. On Thursday, the PI asks which samples were in the allele frequency table shared last month and what filters were applied; answering requires forensic reconstruction from timestamps and notebook entries. By Friday, the data manager has spent more time coordinating files, formats, and provenance than doing science. These are not computational bottlenecks—bcftools [6] can merge VCF files in minutes; PLINK can recode genotypes in seconds—but *coordination* bottlenecks that no existing tool is designed to solve.

The root cause is that population genomics data is inherently relational—variants belong to chromosomes, samples belong to populations, genes carry functional consequences, annotations have versions—yet file-based formats flatten these relationships into disconnected tabular files. Several data management approaches could address this problem (Table 1). Relational databases can store structured metadata but require JOIN operations to reconstruct relationships and do not natively support extensible byte-array properties. Columnar stores (Zarr, HDF5) and distributed frameworks (Hail) excel at large-scale matrix computation but treat annotations, cohorts, and provenance as external metadata. Graph databases offer a complementary approach: data is stored as *nodes* connected by typed *edges*, and queries traverse connections directly. This makes lifecycle operations—incremental addition, annotation versioning, cohort definition, provenance binding—*persistent and local* to the data, rather than requiring regeneration when one relationship changes. Graph databases are not the only viable architecture; the graph-native representation provides a particularly natural fit for the relationship-rich structure of population genomics projects.

Table 1 Comparison of data management architectures for population genomics. Natively supported (•), with external scripting (○), or not supported (—).

Operation	Flat files	Relational DB	Columnar/Hail	GraphMana
Incremental sample addition	—	○	○	•
Annotation versioning	—	○	—	•
Provenance binding	—	○	○	•
Cohort as query	—	•	•	•
Multi-format export (17)	○	—	○	•
In-place annotation update	—	•	—	•
Regeneration avoidance	—	—	○	•

GraphMana exploits graph-native representation to replace the fragmented file collection with a persistent graph database that serves as the project's analytical record (Fig. 1a). The platform organizes 58 CLI commands into nine functional domains—data import, annotation management, data export, sample and cohort management, quality control, provenance tracking, database administration, status reporting, and infrastructure deployment (Supplementary Fig. 1)—providing a complete command-line interface for the data management lifecycle. All operations require no knowledge of Python, graph query languages, or database administration.

The graph schema (Fig. 1b) represents each biallelic variant as a Variant node connected by typed edges to Chromosome, Gene, Sample, and Population nodes. Variant nodes carry a packed genotype array (2 bits per sample, 4 samples per byte) and precomputed population-level allele count, sample count, and frequency arrays. Because these relationships are explicit edges, annotations can be updated by modifying edge properties without touching genotype data, cohorts can be defined as graph traversal queries without extracting files, and provenance is recorded as graph nodes linked to the operations that produced them. This encoding achieves a 125-fold storage reduction compared with per-sample edge representations (Supplementary Table 6) and enables two access paths (Fig. 1c). The FAST PATH reads population arrays directly—constant time regardless of sample count—serving TreeMix [12], site frequency spectra (dadi [10], fastsimcoal2 [11]), BED, and allele frequency tables. The FULL PATH unpacks genotype arrays for per-sample formats (VCF, PLINK 1.9/2.0 [5], EIGENSTRAT [13], Beagle [14], STRUCTURE [15], Genepop, and others) in time linear in N. In total, 17 export formats are produced from the database (Supplementary Note 1). Of these, 6 have been validated against downstream tools, 6 against format specifications, and 5 are functional exports pending external validation (Supplementary Tables 8–9).

Each export automatically generates a JSON manifest sidecar recording the software version, timestamp, output format, variant and sample counts, and all active filter parameters, enabling any collaborator to verify or reproduce the extraction (Supplementary Fig. 2).

Incremental sample addition—the Monday crisis—appends new genotypes to existing packed arrays without modifying previous data (Fig. 1d). On the 1000 Genomes Project [3] dataset (70.7 million variants, 3,202 human whole-genome samples, 26 populations), adding 234 samples takes 182 minutes (Supplementary Table 5), and

no downstream result is invalidated. Annotation updates complete in 3.5 seconds for 53,000 BED regions versus 96 seconds for a full VCF rewrite with bcftools—a 27-fold speedup (Fig. 2b)—because edge properties are modified directly. Format conversion is a single command: `graphmana export --format eigenstrat --populations EUR --filter-maf-min 0.05`, with provenance recorded automatically.

We benchmarked GraphMana against bcftools on a 1000 Genomes Project chromosome 22 workload (1.07 million variants, 3,202 samples) comprising incremental addition, cohort extraction, multi-format export, annotation update, and a lifecycle simulation (Fig. 2; Supplementary Tables 1–3). The comparison reveals a fundamental trade-off. For VCF-centric operations, bcftools is 3–5 times faster per operation (cohort extraction: 49–110 s vs. 131–571 s), reflecting graph database query overhead versus optimized file streaming (Fig. 2b). However, the lifecycle simulation exposes the coordination advantage: GraphMana completed all 46 planned operations in 98 minutes from a persistent database, whereas bcftools completed 17 of 26 in 17 minutes, with 9 operations having no equivalent (Fig. 2a). GraphMana's value lies in avoiding repeated regeneration and manual coordination: each new export, annotation update, or cohort extraction operates on the persistent database without preceding file conversion.

GraphMana currently focuses on biallelic variant representation; multi-allelic VCF records are decomposed into K biallelic nodes at import (Supplementary Note 2). Structural variants are stored with typed properties; gVCF reference blocks are imported as variant nodes, and users may wish to filter them with `--filter-variant-type` to exclude non-variant records. The platform operates as a single-user write system; multi-user read access is supported via read replicas.

GraphMana's packed encoding scales linearly with sample count, and the maximum dataset size is determined by available hardware rather than an architectural ceiling. The scaling estimates in this work are based on human whole-genome sequencing data (∼85 million biallelic variants per genome; Supplementary Table 6); for species or designs with fewer variants (e.g., exome sequencing at ∼5M variants, or rice at ∼29M variants), the storage and time requirements scale proportionally. At 100–10,000 human whole-genome samples, all operations are interactive. Beyond 10,000 samples, FAST PATH operations remain efficient because population arrays are constant size regardless of N, but FULL PATH exports slow proportionally. At biobank scale (500,000+ samples), the single-node graph database architecture becomes a practical bottleneck, and distributed frameworks such as Hail [4] are better suited. The current implementation uses Neo4j Community Edition [2] (free, open source); the graph-native design is not tied to a specific vendor. GraphMana addresses the data lifecycle layer while the companion GraphPop engine [1] provides graph-native analytical computation on the same persistent database. Together, the two platforms provide a persistent analytical record in which project data and computed results are co-located, queryable, and reproducible. Documentation (11 vignettes, 65 command reference pages) and pre-built databases are available at the project repository under the MIT license.

Online Methods

Graph database technology for genomic data management

Population genomics data is inherently a network of typed relationships: variants are located on chromosomes, samples belong to populations, genes carry functional consequences, and every operation has provenance. Conventional approaches store this network as flat files requiring manual relationship maintenance. A graph database stores data as nodes and edges with properties, traversing pre-materialized connections in constant time per hop. This property-graph model maps onto population genomics: a Variant node connects to Chromosome, Gene, and Population nodes via typed edges. Edges can be updated independently—an annotation update modifies edge properties without touching genotype data. A detailed introduction to graph database concepts for non-specialist readers is provided in Supplementary Note 4.

We note that graph databases are one of several possible architectures (Table 1). The graph-native approach was chosen because it co-locates relationship semantics with the data, making lifecycle operations inherently persistent. This representation also serves as the foundation for the companion GraphPop engine [1]. GraphMana uses Neo4j Community Edition [2] as its graph database engine; the design principles are not specific to Neo4j.

Software architecture

GraphMana is implemented as a Python command-line application (21,267 lines) with a Java plugin (2,864 lines) for server-side procedures. The Python layer handles VCF parsing via cyvcf2 [8], genotype packing via NumPy (complementing scikit-allel [7]), CSV generation, and all export format writers. The 58 CLI commands are organized into nine functional domains (Supplementary Fig. 1). A companion Python package provides a Jupyter-compatible API. The test suite comprises 1,439 unit and integration tests.

Graph schema and data encoding

The graph contains five node types: Variant, Sample, Population, Chromosome, and Gene, connected by typed relationships (ON_CHROMOSOME, IN_POPULATION, HAS_CONSEQUENCE, NEXT). Variant nodes store genotype data as packed byte arrays: gt_packed (2 bits per sample), phase_packed (1 bit per sample), and ploidy_packed (1 bit per sample). This stores 4 genotypes per byte, achieving a 125-fold reduction versus per-sample edges (Supplementary Table 6). Each Variant also carries K-element population arrays (ac, an, af, het_count, hom_alt_count, het_exp) that are constant size regardless of N. Full encoding details are provided in Supplementary Note 2.

Import pipeline

GraphMana uses a two-step split pipeline. The prepare-csv step parses VCFs, packs genotypes, computes population statistics, and writes CSV files (embarrassingly parallel, no database needed). The load-csv step uses the database engine's bulk import. On our benchmark hardware (32 cores, 64 GB RAM, NVMe SSD), CSV generation for 2,500 human whole-genome samples across 22 chromosomes (70.7M variants) completed in 95 minutes with 16 threads; bulk loading completed in 3 minutes.

Incremental sample addition

New samples are added by extending packed genotype arrays. Three strategies are provided: Cypher transactions, export-extend-reimport, and CSV-to-CSV rebuild. The CSV-to-CSV path extended 70.7M variants with 234 samples in 182 minutes; approximately 95% required only HomRef extension.

Correctness validation

We validated genotype fidelity by roundtrip (import → VCF export → comparison), achieving 99.999%+ concordance. A systematic correctness matrix is in Supplementary Tables 10–11.

Benchmark methodology

All benchmarks used the 1000 Genomes Project [3] (GRCh38). Chromosome 22 (1,066,557 variants, 3,202 samples, 26 populations) served as the primary target. We compared against bcftools [6] (v1.17) across five benchmarks. Per-operation timings favor bcftools; GraphMana's advantage lies in avoiding repeated regeneration.

Scalability

Storage per variant is $\lceil N/4 \rceil$ bytes for genotypes and $\lceil N/8 \rceil$ bytes for phase. All scaling estimates assume human whole-genome data (∼85M biallelic variants); for species or designs with fewer variants, requirements scale proportionally (Supplementary Tables 6–7). At biobank scale, distributed frameworks are more appropriate.

Deployment

GraphMana runs on workstations and HPC cluster nodes via a two-step split pipeline. SLURM and PBS job scripts are provided. The database must reside on local SSD.

Data availability

GraphMana is available at https://github.com/jfmao/GraphMana under the MIT license. Benchmark data and pre-built databases are deposited at Zenodo (https://doi.org/10.5281/zenodo.19472836).

Code availability

All source code is at https://github.com/jfmao/GraphMana under the MIT license.

Acknowledgements. This work was supported by [funding sources to be completed]. The 1000 Genomes Project dataset [3] was used under its open data access terms.

Author contributions. J.M. conceived the project, designed and implemented the software, performed benchmarks, and wrote the manuscript.

Competing interests

The author declares no competing interests.

References

[1] Mao, J. GraphPop: graph-native population genomics with annotation-conditioned queries. *Nature Methods* (companion paper, in preparation).

[2] Neo4j, Inc. Neo4j Graph Database. https://neo4j.com (Community Edition 5.x).

[3] Byrska-Bishop, M. et al. High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell* **185**, 3426–3440.e19 (2022).

[4] Hail Team. Hail: An Open-Source Framework for Scalable Genomic Data Analysis. https://hail.is.

[5] Chang, C.C. et al. Second-generation PLINK: rising to the challenge of larger and richer datasets. *GigaScience* **4**, s13742-015-0047-8 (2015).

[6] Danecek, P. et al. Twelve years of SAMtools and BCFtools. *GigaScience* **10**, giab008 (2021).

[7] Miles, A. et al. scikit-allel: A Python package for exploring and analysing genetic variation data. https://github.com/cggh/scikit-allel.

[8] Pedersen, B.S. & Quinlan, A.R. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics* **33**, 1867–1869 (2017).

[9] Danecek, P. et al. The variant call format and VCFtools. *Bioinformatics* **27**, 2156–2158 (2011).

[10] Gutenkunst, R.N. et al. Inferring the joint demographic history of multiple populations from multidimensional SNP frequency data. *PLoS Genetics* **5**, e1000695 (2009).

[11] Excoffier, L. et al. fastsimcoal2: demographic inference under complex evolutionary scenarios. *Bioinformatics* **37**, 4882–4885 (2021).

[12] Pickrell, J.K. & Pritchard, J.K. Inference of population splits and mixtures from genome-wide allele frequency data. *PLoS Genetics* **8**, e1002967 (2012).

[13] Patterson, N., Price, A.L. & Reich, D. Population structure and eigenanalysis. *PLoS Genetics* **2**, e190 (2006).

[14] Browning, S.R. & Browning, B.L. Rapid and accurate haplotype phasing and missing-data inference for whole-genome association studies by use of localized haplotype clustering. *Am. J. Hum. Genet.* **81**, 1084–1097 (2007).

[15] Pritchard, J.K., Stephens, M. & Donnelly, P. Inference of population structure using multilocus genotype data. *Genetics* **155**, 945–959 (2000).

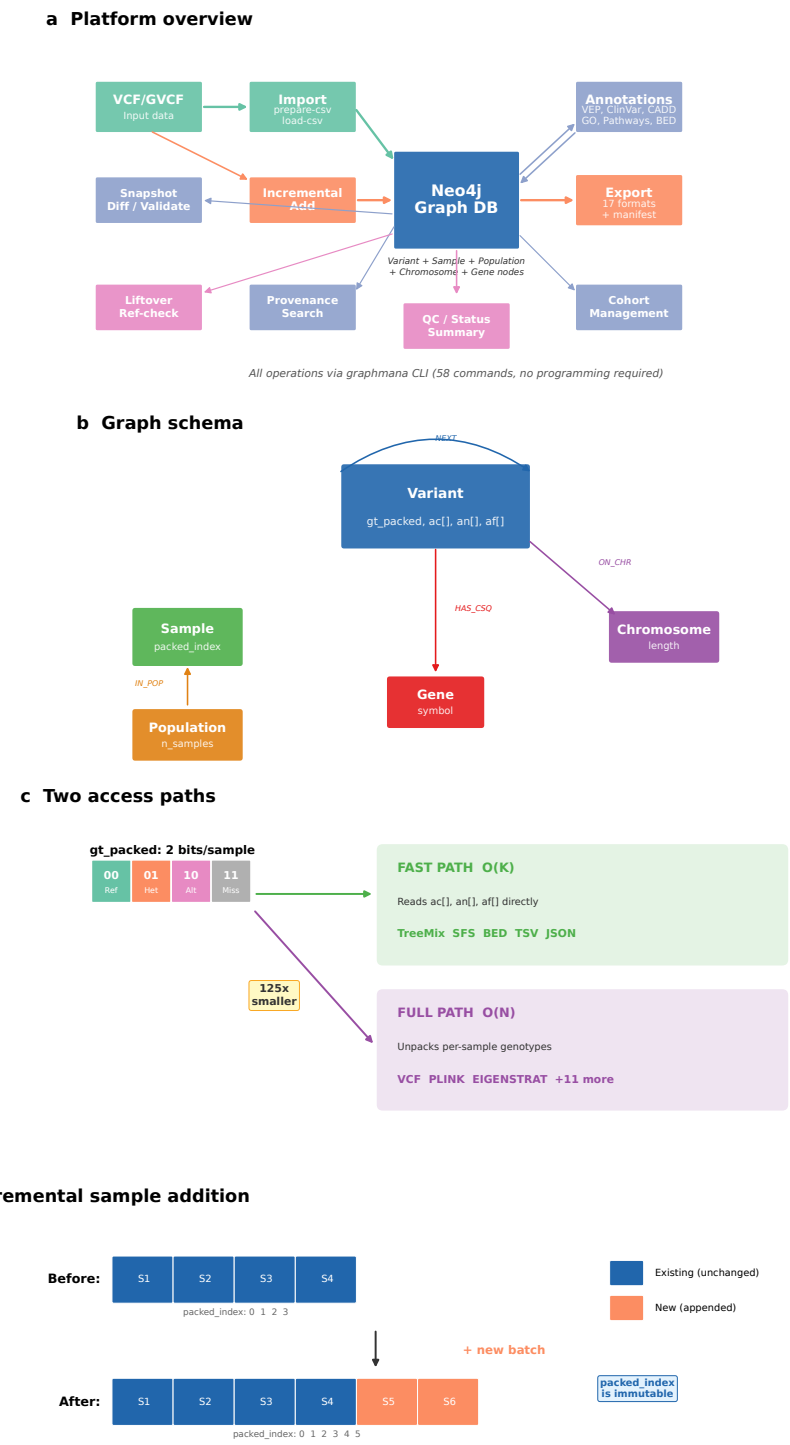

Fig. 1 GraphMana platform architecture. (**a**) System overview showing data flow from VCF input through a graph database to 17 export formats. Functional modules include annotation management, cohort definitions, provenance tracking, quality control, and database administration. All 58 CLI commands are organized into nine functional domains (Supplementary Fig. 1). (**b**) Graph schema: five node types connected by typed relationships. Annotation updates modify edge properties independently of genotype data. (**c**) Two access paths: FAST PATH queries pre-computed population arrays in $O(K)$ time; FULL PATH unpacks per-sample genotypes in $O(N)$ time. (**d**) Incremental sample addition: new genotypes appended to packed arrays; immutable packed_index ensures downstream reference stability.

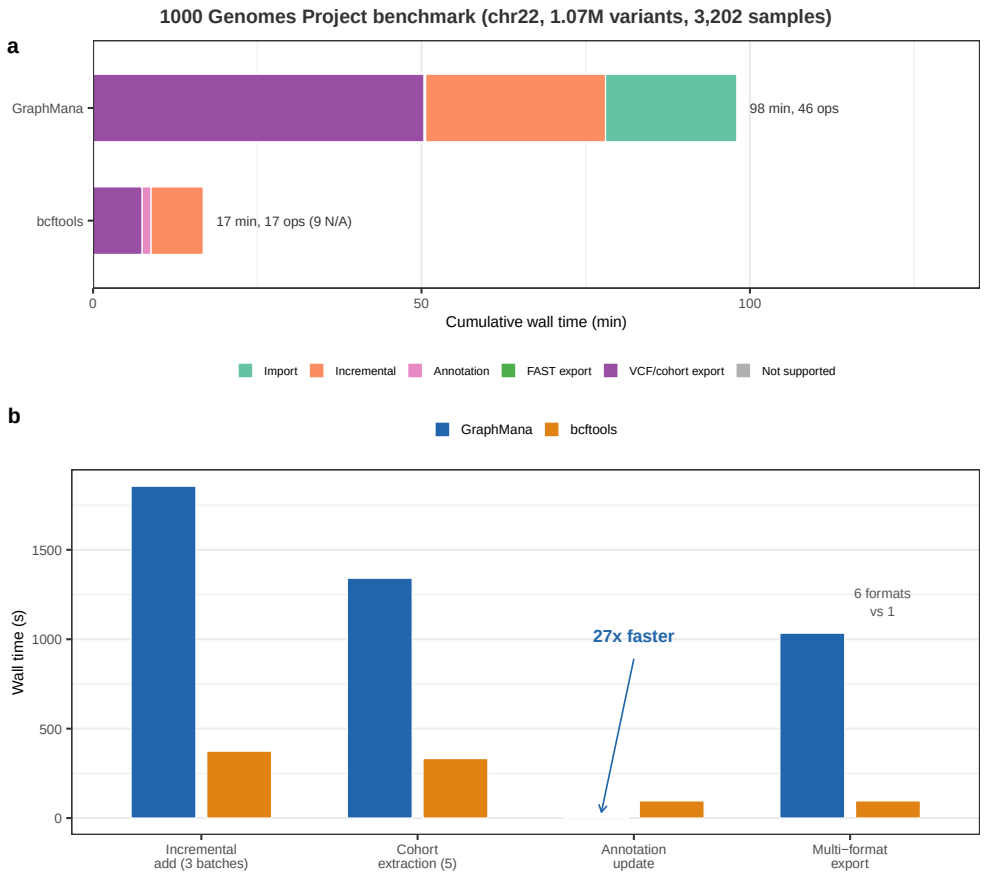

Fig. 2 Benchmark comparison on 1000 Genomes Project chromosome 22 (1.07M variants, 3,202 human whole-genome samples). (**a**) Lifecycle simulation: GraphMana completes 46 operations from a persistent database; bcftools completes 17 of 26 (9 require additional tools, shown in grey). Time difference reflects graph database query overhead offset by elimination of format regeneration. (**b**) Per-task comparison. bcftools is faster for VCF-centric operations. GraphMana achieves a 27-fold speedup for annotation updates through in-place edge modification, and provides multi-format export (6 formats) where bcftools supports only VCF.